GroveLEDBarPWM — Full API Reference

Version: 1.8.0
Hardware: Seeed Grove LED Bar V2.1 / MY9221
Target: Arduino-compatible boards, including ESP32-C3

1. Overview

`GroveLEDBarPWM` provides PWM brightness control for all ten LEDs on the Grove LED Bar V2.1. It also provides:

- Individual LED brightness
- Brightness arrays
- Graduated brightness
- Green-to-red logical ordering
- Smooth non-blocking transitions
- Individual LED flashing
- Moving Dot effect
- Knight Rider effect
- Pulse/Breathe effect
- Configurable data and clock pins

The library is designed around a non-blocking `update()` function, allowing the LED bar animation to run while the main application continues to handle sensors, MQTT, LoRa, Wi-Fi, and other tasks.

---

2. Installation

Copy the `GroveLEDBarPWM` library into your Arduino libraries directory.

Then include:

```
#include <GroveLEDBarPWM.h>
```

---

3. Hardware and Pins

The constructor accepts the DATA and CLOCK GPIO pins:

```
GroveLEDBarPWM bar(DATA_PIN, CLOCK_PIN);
```

Example:

```
GroveLEDBarPWM bar(20, 21);
```

The pin numbers are not fixed by the library.

For example:

```
GroveLEDBarPWM bar(7, 6);
```

is also valid when those GPIOs are connected correctly.

---

4. Grove LED Bar Channel Mapping

The MY9221 channel order is different from the logical LED order used by the library.

The physical channel mapping is:

MY9221 Channel	LED colour
0	Red
1	Yellow
2	Green
3	Green
4	Green
5	Green
6	Green
7	Green
8	Green
9	Green

When green-to-red ordering is enabled, the library presents the bar logically from green to red.

Logical LED numbers are **zero based**:

```
LED 0 = first logical LED
LED 9 = last logical LED
```

---

5. Constructor

`GroveLEDBarPWM()`

Syntax

```
GroveLEDBarPWM(uint8_t dataPin, uint8_t clockPin);
```

Example

```
GroveLEDBarPWM bar(20, 21);
```

Creates an LED bar object using GPIO 20 for DATA and GPIO 21 for CLOCK.

---

6. Initialisation

`begin()`

Initialises the DATA and CLOCK pins and clears the LED bar.

Syntax

```
bar.begin();
```

Call this from `setup()`.

---

7. Pin Configuration

`setPins()`

Changes the DATA and CLOCK GPIO pins.

```
bar.setPins(dataPin, clockPin);
```

Example:

```
bar.setPins(7, 6);
```

`getDataPin()`

Returns the currently configured DATA GPIO.

```
uint8_t pin = bar.getDataPin();
```

`getClockPin()`

Returns the currently configured CLOCK GPIO.

```
uint8_t pin = bar.getClockPin();
```

---

8. Basic Bar Control

`setLevel()`

Sets the number of logical LEDs that should be illuminated.

```
bar.setLevel(level);
```

Example:

```
bar.setLevel(5);
```

With transitions disabled, the requested level is applied immediately.

With transitions enabled, the bar moves smoothly toward the requested level.

Valid range:

```
0 to 10
```

`clear()`

Turns all LEDs off and stops the current effect.

```
bar.clear();
```

`show()`

Sends the current brightness frame to the LED bar.

```
bar.show();
```

---

9. Individual LED Brightness

`setBrightness()`

Sets an individual LED's PWM value.

```
bar.setBrightness(index, brightness);
```

Brightness range:

```
0 to 255
```

Example:

```
bar.setBrightness(5, 128);
```

sets logical LED 5 to approximately 50% PWM.

`setBrightnessPercent()`

Sets an individual LED using a percentage.

```
bar.setBrightnessPercent(index, percent);
```

Example:

```
bar.setBrightnessPercent(5, 50);
```

`getBrightness()`

Returns the current PWM brightness of a logical LED.

```
uint8_t value = bar.getBrightness(5);
```

Returns:

```
0 to 255
```

`getBrightnessPercent()`

Returns the current brightness as a percentage.

```
uint8_t value = bar.getBrightnessPercent(5);
```

---

10. Brightness Arrays

`setBrightnessArray()`

Sets all ten LEDs using percentages.

```
uint8_t brightness[10] = {
  10, 20, 30, 40, 50,
  60, 70, 80, 90, 100
};

bar.setBrightnessArray(brightness);
```

Each value is:

```
0 to 100 percent
```

---

getBrightnessArray()

Returns the current brightness of all ten LEDs as percentages.

```
uint8_t brightness[10];

bar.getBrightnessArray(brightness);
```

---

setBrightnessArrayPWM()

Sets all ten LEDs directly using PWM values.

```
uint8_t brightness[10] = {
  25, 50, 75, 100, 125,
  150, 175, 200, 225, 255
};

bar.setBrightnessArrayPWM(brightness);
```

Values range from:

```
0 to 255
```

---

getBrightnessArrayPWM()

Returns all ten LED brightness values as PWM values.

```
uint8_t brightness[10];

bar.getBrightnessArrayPWM(brightness);
```

---

setAllBrightness()

Sets all ten LEDs to the same PWM value.

```
bar.setAllBrightness(128);
```

---

11. Graduated Brightness

setGraduated()

Enables or disables graduated brightness when using levels and transitions.

```
bar.setGraduated(true);
```

Disable:

```
bar.setGraduated(false);
```

The graduated curve used by the library is:

```
5% → 10% → 20% → 30% → 40%
→ 50% → 60% → 70% → 80% → 100%
```

---

getGraduated()

Returns whether graduated brightness is enabled.

```
bool enabled = bar.getGraduated();
```

---

setGraduationMin()

Sets the minimum graduation percentage.

```
bar.setGraduationMin(5);
```

The value is constrained to 0–100%.

---

getGraduationMin()

Returns the configured graduation minimum.

```
uint8_t value = bar.getGraduationMin();
```

---

12. Green-to-Red Direction

setGreenToRed()

Controls the logical direction of the bar.

```
bar.setGreenToRed(true);
```

With this enabled, logical LED numbering follows the intended green-to-red display direction.

Disable it with:

```
bar.setGreenToRed(false);
```

---

getGreenToRed()

Returns the current direction setting.

```
bool enabled = bar.getGreenToRed();
```

---

13. Smooth Transitions

Transitions are non-blocking.

setTransition()

Enable or disable smooth level transitions.

```
bar.setTransition(true);
```

Disable:

```
bar.setTransition(false);
```

---

getTransition()

Returns whether transitions are enabled.

```
bool enabled = bar.getTransition();
```

---

setTransitionSpeed()

Sets the transition speed.

```
bar.setTransitionSpeed(10);
```

Higher values produce faster brightness changes.

---

getTransitionSpeed()

Returns the transition speed.

```
uint8_t speed = bar.getTransitionSpeed();
```

---

setTransitionTime()

Sets the approximate transition time.

```
bar.setTransitionTime(500);
```

The value is in milliseconds.

---

getTransitionTime()

Returns the configured transition time.

```
uint16_t time = bar.getTransitionTime();
```

---

isTransitioning()

Returns `true` while the bar is still moving toward its requested level.

```
if (bar.isTransitioning()) {
  // Transition still running
}
```

---

14. Flashing an Individual LED

setFlashSpeed()

Sets the flash timing.

```
bar.setFlashSpeed(500);
```

The value is in milliseconds.

---

getFlashSpeed()

Returns the configured flash speed.

```
uint16_t speed = bar.getFlashSpeed();
```

---

flashLED()

Starts a flash on one logical LED.

```
bar.flashLED(5);
```

The LED rises from its current brightness to full brightness and then returns toward the current underlying brightness.

The flash is non-blocking.

`isFlashing()`

Returns `true` while an LED flash is active.

```
if (bar.isFlashing()) {
  // Flash is active
}
```

---

15. Effects

Effects are controlled using the `Effect` enumeration.

Available effects in V1.8.0:

```
GroveLEDBarPWM::EFFECT_NONE
GroveLEDBarPWM::EFFECT_MOVING_DOT
GroveLEDBarPWM::EFFECT_KNIGHT_RIDER
GroveLEDBarPWM::EFFECT_PULSE
```

`setEffectSpeed()`

Sets the effect speed.

```
bar.setEffectSpeed(150);
```

The value is in milliseconds between effect updates.

`getEffectSpeed()`

Returns the configured effect speed.

```
uint16_t speed = bar.getEffectSpeed();
```

`startEffect()`

Starts an effect.

Moving Dot

```
bar.startEffect(GroveLEDBarPWM::EFFECT_MOVING_DOT);
```

The dot travels:

`LED 0 → LED 9 → LED 0`

Knight Rider

```
bar.startEffect(GroveLEDBarPWM::EFFECT_KNIGHT_RIDER);
```

The centre LED is brightest with graduated neighbouring LEDs.

Pulse/Breathe

```
bar.startEffect(GroveLEDBarPWM::EFFECT_PULSE);
```

All ten LEDs smoothly brighten and dim together.

`stopEffect()`

Stops the current effect.

```
bar.stopEffect();
```

The underlying bar brightness is restored.

`isEffectRunning()`

Returns `true` when an effect is active.

```
if (bar.isEffectRunning()) {
  // Effect running
}
```

---

16. Pulse/Breathe

`setPulseRange()`

Sets the minimum and maximum brightness of the Pulse/Breathe effect.

```
bar.setPulseRange(minimumPercent, maximumPercent);
```

Example:

```
bar.setPulseRange(10, 80);
```

This produces:

`10% → 80% → 10% → 80% ...`

Values are constrained to 0–100%.

`getPulseMinimum()`

Returns the configured minimum percentage.

```
uint8_t minimum = bar.getPulseMinimum();
```

`getPulseMaximum()`

Returns the configured maximum percentage.

```
uint8_t maximum = bar.getPulseMaximum();
```

---

17. The `update()` Function

`update()`

This is the main function for all non-blocking transitions, flashes and effects.

```
bar.update();
```

It should normally be called repeatedly from `loop()`.

Example:

```
void loop()
{
  bar.update();

  // Other application code can run here.
}
```

Do not replace `update()` with a long `delay()` if you want the effect to remain smooth and responsive.

---

18. Complete Example

The following example demonstrates a typical application:

```ino
#include <GroveLEDBarPWM.h>

GroveLEDBarPWM bar(20, 21);

void setup()
{
  bar.begin();

  bar.setGreenToRed(true);
  bar.setGraduated(true);

  bar.setTransitionTime(500);
  bar.setLevel(5);
}

void loop()
{
  bar.update();

  // Other application code can run here.
}
```

---

19. Process Indicator Example

A useful application is to use Moving Dot while a process is running:

```ino
bar.setEffectSpeed(150);
bar.startEffect(GroveLEDBarPWM::EFFECT_MOVING_DOT);

while (processRunning) {
  bar.update();

  // Process code...
}

bar.stopEffect();
```

For a Knight Rider process indicator:

```ino
bar.setEffectSpeed(100);
bar.startEffect(GroveLEDBarPWM::EFFECT_KNIGHT_RIDER);
```

For an idle/waiting indicator:

```ino
bar.setPulseRange(10, 60);
bar.setEffectSpeed(20);
```

```
bar.startEffect(GroveLEDBarPWM::EFFECT_PULSE);
```

---

20. Non-Blocking Design

The library's transitions, flashing and effects are designed to be non-blocking.

The recommended pattern is:

```
void loop()
{
  bar.update();

  readSensors();
  handleMQTT();
  handleLoRa();
  checkButtons();
}
```

This allows the LED bar to animate without preventing other application code from running.

---

21. API Summary

Function	Purpose
`begin()`	Initialise the LED bar
`setPins()`	Change DATA/CLOCK pins
`getDataPin()`	Read DATA pin
`getClockPin()`	Read CLOCK pin
`setLevel()`	Set bar level
`clear()`	Turn all LEDs off
`show()`	Send current frame
`setBrightness()`	Set one LED PWM
`setBrightnessPercent()`	Set one LED percentage
`getBrightness()`	Read one LED PWM
`getBrightnessPercent()`	Read one LED percentage
`setBrightnessArray()`	Set all LEDs by percentage
`getBrightnessArray()`	Read all LEDs by percentage
`setBrightnessArrayPWM()`	Set all LEDs by PWM
`getBrightnessArrayPWM()`	Read all LEDs by PWM
`setAllBrightness()`	Set all LEDs equally
`setGraduated()`	Enable/disable graduation
`getGraduated()`	Read graduation state

Function	Purpose
`setGraduationMin()`	Set graduation minimum
`getGraduationMin()`	Read graduation minimum
`setGreenToRed()`	Set logical direction
`getGreenToRed()`	Read logical direction
`setTransition()`	Enable/disable transitions
`getTransition()`	Read transition state
`setTransitionSpeed()`	Set transition speed
`getTransitionSpeed()`	Read transition speed
`setTransitionTime()`	Set transition time
`getTransitionTime()`	Read transition time
`isTransitioning()`	Check transition status
`setFlashSpeed()`	Set flash speed
`getFlashSpeed()`	Read flash speed
`flashLED()`	Flash one LED
`isFlashing()`	Check flash status
`setEffectSpeed()`	Set effect speed
`getEffectSpeed()`	Read effect speed
`startEffect()`	Start an effect
`stopEffect()`	Stop an effect
`isEffectRunning()`	Check effect status
`setPulseRange()`	Set Pulse/Breathe range
`getPulseMinimum()`	Read pulse minimum
`getPulseMaximum()`	Read pulse maximum
`update()`	Run non-blocking animation

---

22. Version History

V1.0

Basic PWM LED control.

V1.1

Graduated brightness.

V1.2

Configurable DATA/CLOCK pins and smooth transitions.

V1.3

Individual LED and array brightness control.

V1.4

Individual LED flashing.

V1.5

Independent flash overlay and improved flash/transition interaction.

V1.6

Moving Dot effect.

V1.7

Knight Rider effect.

V1.8

Pulse/Breathe effect with configurable minimum and maximum brightness.

End of API Reference

GroveLEDBarPWM V1.8.0